

Take Home Task Bulk URL Health Checker

Duration: 3 days

Deliverables: GitHub repo + 3–5 minute Loom walkthrough

What to Build

A dashboard where a user submits a list of URLs. The backend checks each URL in the background and records its result. The UI reflects progress as results arrive.

For each URL, capture at minimum: final HTTP status code, response time, and page title where one exists.

Required Stack

- Node.js + TypeScript
- Fastify
- PostgreSQL
- Redis
- BullMQ
- Next.js + TypeScript

Everything else is your call: schema design, live update transport, query layer, container setup, project structure.

Functional Requirements

1. Batch submission

- The user pastes a list of URLs or uploads a CSV.
- The batch and its URLs are persisted in PostgreSQL before any checking begins.
- Each URL is processed as its own background job.
- The response gives the client whatever it needs to track that batch.

2. Background processing

Workers run in a **separate process** from the API server, and must satisfy:

- **Global rate limit:** 10 requests/second across the entire system — not per URL, not per worker process.
- **Concurrency:** 5 checks in flight.
- **Retries:** up to 3 on transient failure, with exponential backoff.

These guarantees must still hold if more than one worker process is running.

3. Live updates

- As each URL finishes, the batch page updates and the progress indicator moves without user action.
- **Refresh safe:** reloading mid batch must produce complete and correct state.
- Must remain correct when more than one API instance is serving clients.
- The client must recover correctly from a dropped connection.

The transport mechanism is your choice. Be ready to defend it.

4. UI

- A view listing all batches, and a view for a single batch.
- Each batch is addressable by its own URL, so it can be linked to or opened directly.
- Opening a batch URL cold — new tab, no prior client state — must produce the correct state, whether that batch is still running or already finished.

5. Controls

- **Cancel batch** — must behave correctly against both queued jobs and jobs already in flight.
- **Retry failed only** — re runs only the URLs that ended in a failed state, without re doing successful work.

In both cases, the persisted state must stay consistent with what the user sees.

6. Caching

The batch list endpoint must be served from a 30 second cache. Cached data must not go stale in a user visible way when a batch is created or changes state.

7 Type safety

Shared types between client and server.

8. README

- The exact command that runs the whole system.
 - Architecture overview.
 - Trade offs you made, and what you would do differently with more time.
 - How the system behaves if the API is scaled horizontally.
-

Out of Scope

Auth, notifications, charts, polished UI. We are not evaluating visual design — function over form.

How We Evaluate

- Whether the rate limit, concurrency, and retry requirements actually hold in practice.
 - Why each piece of infrastructure is in your design, and what breaks without it.
 - Where the source of truth for batch and job state lives.
 - Idempotency.
 - Type safety across the client/server boundary.
 - Command of Next.js fundamentals: routing, the server/client boundary, and where and when data is fetched and whether those choices look deliberate or default.
 - Separation between the API process, the worker process, and the UI.
 - Resilience of the live update path.
 - Clarity of the README and the Loom.
-

Deliverables

1. **GitHub repo** that runs with one command, documented in the README.
 2. **Loom, 3–5 minutes**, covering: your architecture, your infrastructure choices, your retry and idempotency strategy, your live update transport choice, and the trade-offs you made under time pressure.
-

Before You Start

If something is ambiguous, we would rather answer a question than have you build the wrong thing. If you choose to make an assumption instead, record it in the README.